Review Intelligence System

Technical Design Report

Backend Engineering Challenge · MindBugs

May 6, 2026

Overview & Motivation

Hotel managers receive large volumes of unstructured customer reviews that are difficult to analyze at scale. The Review Intelligence System addresses this by providing a fully automated pipeline that transforms raw review text into structured, deduplicated insights — surfacing what guests value most and which complaints recur — through a single API call.

The system is designed for **local, self-contained deployment**: all inference runs on-premise using Ollama, no external API keys are required, and the entire stack is reproducible with a single `docker compose up -build` command. This design choice reflects a deliberate trade-off: lower output quality in exchange for privacy, cost predictability, and zero runtime dependencies on third-party services.

System Architecture

The system is composed of four Docker services and four Python modules, each with a single, well-defined responsibility.

Service Layer

Service	Image	Role
`ollama`	`ollama/ollama:latest`	LLM inference runtime, exposes REST API on port 11434
`ollama-pull`	`ollama/ollama:latest`	Init container; pulls `llama3.2:1b` once, then exits
`app`	Custom (Python 3.11-slim)	FastAPI backend; orchestrates the full pipeline
`frontend`	`nginx:alpine`	Serves the UI and proxies `/api/` to the backend

Startup order is enforced by Docker Compose dependency conditions: `ollama` must pass its healthcheck before `ollama-pull` runs, and `ollama-pull` must exit with code 0 before `app` starts. This *init container pattern* ensures the LLM model is fully downloaded before the API accepts any traffic, producing a clear startup failure rather than a silent runtime error.

Processing Pipeline

The `/process_file` endpoint executes seven stages in sequence:

1. **Ingest** — Read all reviews from `/data/reviews.txt`; strip whitespace and filter empty lines.
2. **Extraction** — For each review, call `llama3.2:1b` via Ollama to extract highlights and pain points as structured JSON. All requests are dispatched concurrently using `asyncio.gather()`.
3. **Flatten & filter** — Aggregate all extracted highlights into one list and all pain points into another; discard any empty strings produced by the LLM.

4. **Embedding** — Embed all highlights and all pain points separately using the pre-loaded `SentenceTransformer` model.

5. **Cross-validation** — Remove pain points whose cosine similarity to any highlight exceeds 0.70. This catches items the LLM misclassified as negative when they are semantically positive (e.g. "very comfortable beds" appearing in the pain-points list).

6. **Deduplication** — Pairwise cosine similarity is computed for each list independently; items above the 0.8 threshold are merged into clusters via greedy Union-Find. Each cluster elects a centroid representative. A full analysis report is written to `/data/output/analysis.txt`.

7. **Response** — Deduplicated highlights and pain points are returned as JSON, each entry carrying a `count` field indicating how many semantically equivalent items it consolidated.

Key Technical Decisions

Embedding Model: `multi-qa-MiniLM-L6-cos-v1`

The choice of embedding model has a direct and significant impact on deduplication quality. The selected model, `multi-qa-MiniLM-L6-cos-v1`, was trained explicitly on question–answer pairs for *semantic similarity under cosine distance* — which is precisely the task at hand. This gives it a marginal but meaningful advantage over the more commonly cited `all-MiniLM-L6-v2`, which was trained on a broader objective. Both models produce 384-dimensional embeddings and have an identical memory footprint (≈90 MB), making the switch cost-free.

Embeddings are L2-normalised at inference time, which reduces cosine similarity to a simple dot product and eliminates a class of floating-point edge cases in the similarity computation.

Structured LLM Output via `format: "json"`

Small models (1B parameters) are prone to ignoring formatting instructions in free-form generation. Rather than relying on regex post-processing — which is fragile and requires its own test coverage — the Ollama API's `format: "json"` parameter is used. This activates constrained decoding, guaranteeing syntactically valid JSON output regardless of prompt adherence. A fallback handler catches any residual parsing failures and logs a warning rather than crashing the pipeline.

Greedy Union-Find Clustering

A naive deduplication approach (remove item j if similarity$(i, j) > \theta$) is order-dependent and may produce inconsistent results. Instead, a Union-Find data structure is used to group all items transitively: if $A \sim B$ and $B \sim C$, all three form one cluster regardless of the order in which pairs are evaluated. This is deterministic, runs in $O(n^2)$ time (acceptable for review-scale data), and requires no additional hyperparameters beyond the similarity threshold.

Centroid-Based Representative Selection

When selecting the representative item for a cluster, the *centroid* strategy is used: the item whose embedding has the highest mean cosine similarity to all other embeddings in the cluster. This is preferred over selecting the first-encountered item because it is order-independent and semantically more central — it minimizes the maximum representational distance to any cluster member.

Concurrent Extraction with `asyncio.gather()`

LLM inference is the dominant latency contributor in the pipeline. Rather than processing reviews sequentially, all extraction calls are dispatched concurrently via `asyncio.gather()`. For a file of n reviews, this reduces wall-clock latency from $O(n \cdot t_{\text{inference}})$ to approximately $O(t_{\text{inference}})$, bounded by Ollama's `OLLAMA_NUM_PARALLEL` setting.

Threshold Sensitivity Analysis

The cosine similarity threshold $\theta = 0.8$ is specified in the requirements. Its effect on deduplication behaviour is non-trivial and warrants explicit documentation.

Threshold θ	Observed behaviour (example corpus)	Risk
0.70	All three pain-point variants collapse into one cluster	Over-merging; distinct com
0.80	Correct grouping on example corpus	(Specified value)
0.85	"Reception process was slow" may remain separate ($\cos \approx 0.82$)	Under-merging; duplicates s
0.95	Near-identical phrasings only; most items remain separate	Deduplication largely ineffe

The optimal threshold is jointly determined by the embedding model's geometry and the linguistic variation in the review corpus. A threshold validated on one domain (hotel reviews in English) should not be assumed to transfer to other domains without re-evaluation.

Limitations & Future Improvements

Model quality. `llama3.2:1b` is a constrained model; extraction quality degrades on long, complex, or multilingual reviews. A drop-in replacement with `llama3.2:3b` or `mistral:7b` would improve recall at the cost of increased inference time.

Scalability. The current implementation loads the entire review file into memory and embeds all items in a single batch. For corpora exceeding several thousand reviews, a streaming ingestion approach with batched embedding would be required.

Threshold calibration. The 0.8 threshold is currently a hard-coded constant justified only by the example corpus. A production system should expose this as a configurable parameter and provide tooling to evaluate its effect on a labelled validation set.

Representative quality. The centroid strategy selects the most *average* item in a cluster. A more sophisticated approach would re-prompt the LLM to generate a canonical summary of the cluster, trading latency for output quality.

No persistence. Each API call re-processes the entire file. A caching layer keyed on file hash, or a proper database backend, would be necessary for repeated queries on large corpora.

All code is available in the accompanying GitHub repository. The system was implemented and tested on macOS (Apple Silicon, 8 GB RAM) using Docker Desktop.